```
import re
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.sparse import hstack, csr_matrix

from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import MultinomialNB
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)

sns.set_style("whitegrid")
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True
```

```
df_raw = pd.read_csv("/content/drive/MyDrive/phishing_email_cleaned.csv")
df_raw = df_raw.rename(columns={"text_combined": "email_text"})
df_raw = df_raw.dropna(subset=["email_text"]).reset_index(drop=True)

print("Dataset shape:", df_raw.shape)
print()
print("Class balance:")
print(df_raw["label"].value_counts())
print("(label 1 = phishing, label 0 = legitimate)")
df_raw.head()
```

```
Dataset shape: (82073, 2)

Class balance:
label
1    42840
0    39233
Name: count, dtype: int64
(label 1 = phishing, label 0 = legitimate)
```

	email_text	label
0	hpl nom may 25 2001 see attached file hplno 52...	0
1	nom actual vols 24 th forwarded sabrae zajac h...	0
2	enron actuals march 30 april 1 201 estimated a...	0
3	hpl nom may 30 2001 see attached file hplno 53...	0
4	hpl nom june 1 2001 see attached file hplno 60...	0

Next steps: Generate code with `df_raw` New interactive sheet

```
print("Missing values:")
print(df_raw.isnull().sum())
print()

lengths = df_raw["email_text"].str.len()
print("Email character-length distribution:")
print(lengths.describe())
print()
print("Number of emails longer than 50,000 characters:", (lengths > 50000).sum())
print()
print("Example phishing email (first 300 chars):")
print(df_raw[df_raw.label == 1]["email_text"].iloc[0][:300])
print()
print("Example legitimate email (first 300 chars):")
print(df_raw[df_raw.label == 0]["email_text"].iloc[0][:300])
```

```
Missing values:
email_text    0
label         0
dtype: int64

Email character-length distribution:
count    8.207300e+04
mean     1.290574e+03
std      1.553553e+04
min      1.000000e+01
```

```
25%      2.760000e+02
50%      5.580000e+02
75%      1.339000e+03
max      4.279526e+06
Name: email_text, dtype: float64

Number of emails longer than 50,000 characters: 56

Example phishing email (first 300 chars):
link dwl g 510 802 11 g wireless pci lan adapter 39 85 39 85 dwl g 510 high speed 2 4 ghz 802 11 g wireless pci lan adapter ie

Example legitimate email (first 300 chars):
hpl nom may 25 2001 see attached file hplno 525 xls hplno 525 xls
```

```
MAX_CHARS = 5000

def clean_text(text):
    text = str(text)[:MAX_CHARS]
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    tokens = [w for w in text.split() if w not in ENGLISH_STOP_WORDS]
    return " ".join(tokens)

t0 = time.time()
df_raw["email_text"] = df_raw["email_text"].astype(str).str.slice(0, MAX_CHARS)
df_raw["clean_text"] = df_raw["email_text"].apply(clean_text)
print(f"Cleaned {len(df_raw)} emails in {time.time()-t0:.1f} seconds.")
df_raw[["email_text", "clean_text"]].head()
```

```
Cleaned 82073 emails in 10.7 seconds.
```

	email_text	clean_text
0	hpl nom may 25 2001 see attached file hplno 52...	hpl nom attached file hplno xls hplno xls
1	nom actual vols 24 th forwarded sabrae zajac h...	nom actual vols th forwarded sabrae zajac hou ...
2	enron actuals march 30 april 1 201 estimated a...	enron actuals march april estimated actuals ma...
3	hpl nom may 30 2001 see attached file hplno 53...	hpl nom attached file hplno xls hplno xls
4	hpl nom june 1 2001 see attached file hplno 60...	hpl nom june attached file hplno xls hplno xls

```
URGENT_WORDS = {"urgent", "immediately", "verify", "suspended", "action", "required",
                "click", "confirm", "expire", "expires", "alert", "blocked", "limited"}
URL_TOKENS = {"http", "https", "www"}

def extract_metadata_features(text):
    words = re.findall(r"[a-zA-Z]+", str(text).lower())
    url_count = sum(1 for w in words if w in URL_TOKENS)
    urgent_word_count = sum(1 for w in words if w in URGENT_WORDS)
    return pd.Series({
        "url_count": url_count,
        "urgent_word_count": urgent_word_count,
        "text_length": len(words),
    })

t0 = time.time()
meta_features = df_raw["email_text"].apply(extract_metadata_features)
df_features = pd.concat([df_raw, meta_features], axis=1)
print(f"Extracted metadata features in {time.time()-t0:.1f} seconds.")

df_features[["label", "url_count", "urgent_word_count", "text_length"]].head()
```

```
Extracted metadata features in 21.9 seconds.
```

	label	url_count	urgent_word_count	text_length
0	0	0	0	10
1	0	0	0	152
2	0	0	0	17
3	0	0	0	10
4	0	0	0	10

```
tfidf = TfidfVectorizer(max_features=5000, min_df=3)
X_text = tfidf.fit_transform(df_features["clean_text"])

meta_cols = ["url_count", "urgent_word_count", "text_length"]
X_meta = csr_matrix(df_features[meta_cols].values.astype(float))

X = hstack([X_text, X_meta]).tocsr()
y = df_features["label"].values

feature_names = list(tfidf.get_feature_names_out()) + meta_cols

print("Final feature matrix shape:", X.shape)
```

```
Final feature matrix shape: (82073, 5003)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)
print("Train size:", X_train.shape[0], " Test size:", X_test.shape[0])
```

```
Train size: 65658  Test size: 16415
```

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=RANDOM_STATE),
    "Random Forest": RandomForestClassifier(n_estimators=200, random_state=RANDOM_STATE, n_jobs=-1),
    "Naive Bayes": MultinomialNB(),
    "Neural Network (MLP)": MLPClassifier(hidden_layer_sizes=(32,), max_iter=200,
                                          early_stopping=True, random_state=RANDOM_STATE),
}

trained_models = {}
predictions = {}

for name, model in models.items():
    t0 = time.time()
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    trained_models[name] = model
    predictions[name] = preds
    print(f"Trained {name} in {time.time()-t0:.1f} seconds.")
```

```
Trained Logistic Regression in 24.4 seconds.
Trained Random Forest in 233.7 seconds.
Trained Naive Bayes in 0.0 seconds.
Trained Neural Network (MLP) in 42.1 seconds.
```

```
results = []
for name, preds in predictions.items():
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, preds),
        "Precision": precision_score(y_test, preds),
        "Recall": recall_score(y_test, preds),
        "F1-score": f1_score(y_test, preds),
    })

results_df = pd.DataFrame(results).sort_values("F1-score", ascending=False).reset_index(drop=True)
results_df.to_csv("comparative_analysis_real_data.csv", index=False)
results_df
```

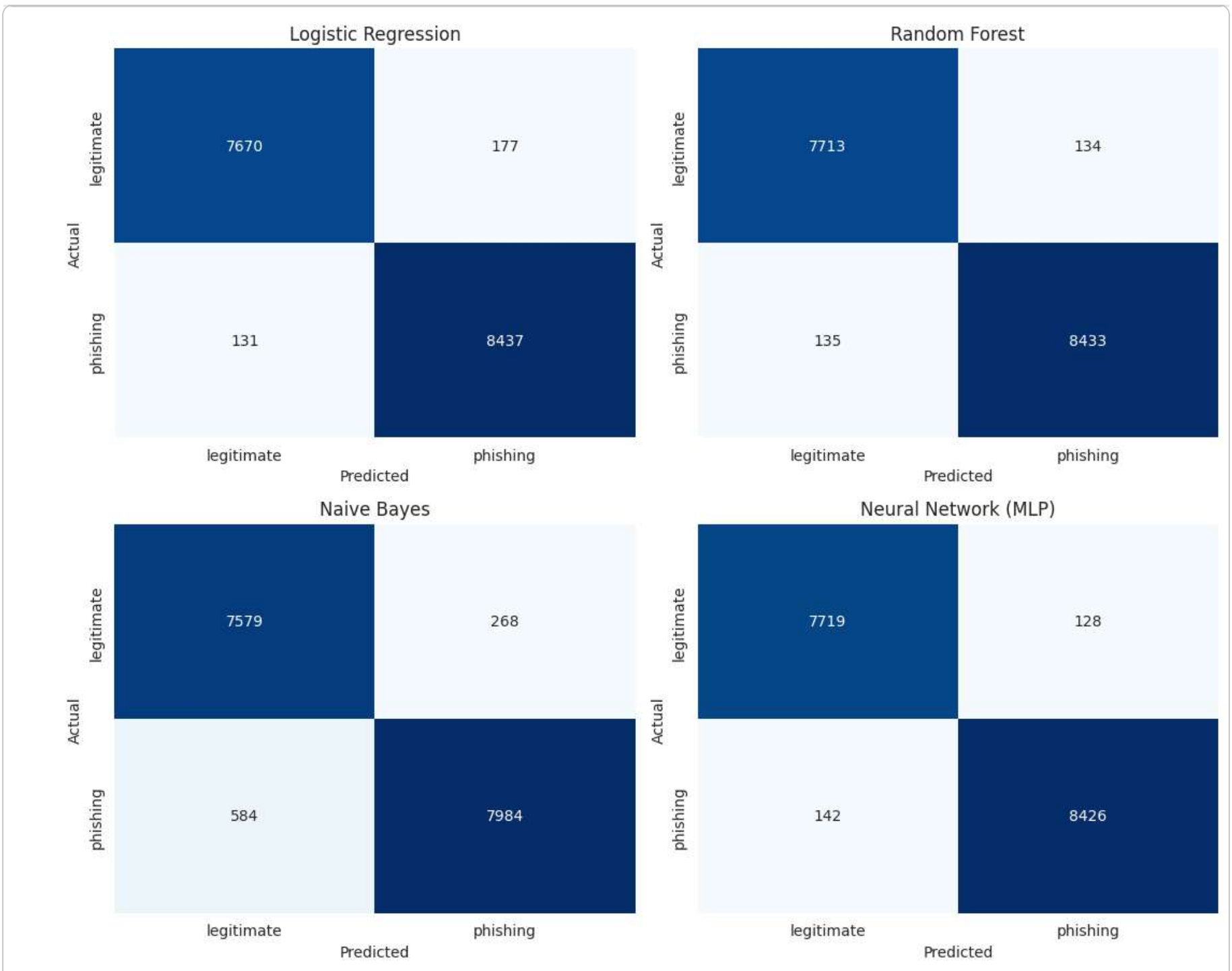
	Model	Accuracy	Precision	Recall	F1-score
0	Random Forest	0.983613	0.984359	0.984244	0.984301
1	Neural Network (MLP)	0.983552	0.985036	0.983427	0.984231
2	Logistic Regression	0.981237	0.979452	0.984711	0.982074
3	Naive Bayes	0.948096	0.967523	0.931839	0.949346

Next steps: Generate code with `results_df` New interactive sheet

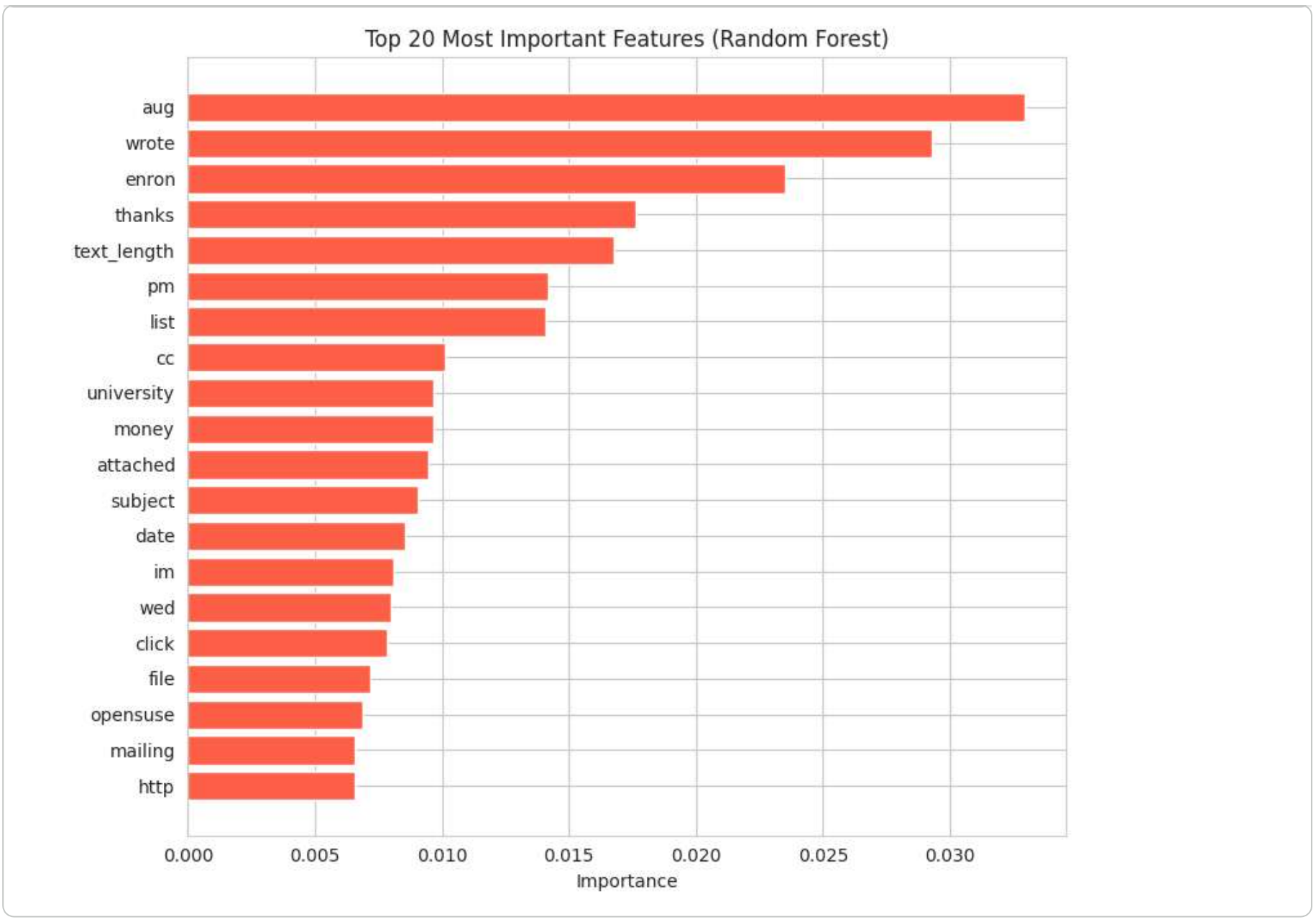
```
best_model_name = results_df.iloc[0]["Model"]
print(f"Best performing model: {best_model_name}\n")
print(classification_report(y_test, predictions[best_model_name],
                            target_names=["legitimate", "phishing"]))
```

```
Best performing model: Random Forest

              precision    recall  f1-score   support

  legitimate       0.98      0.98      0.98      7847
    phishing       0.98      0.98      0.98      8568

    accuracy                           0.98     16415
   macro avg       0.98      0.98      0.98     16415
weighted avg       0.98      0.98      0.98     16415
```

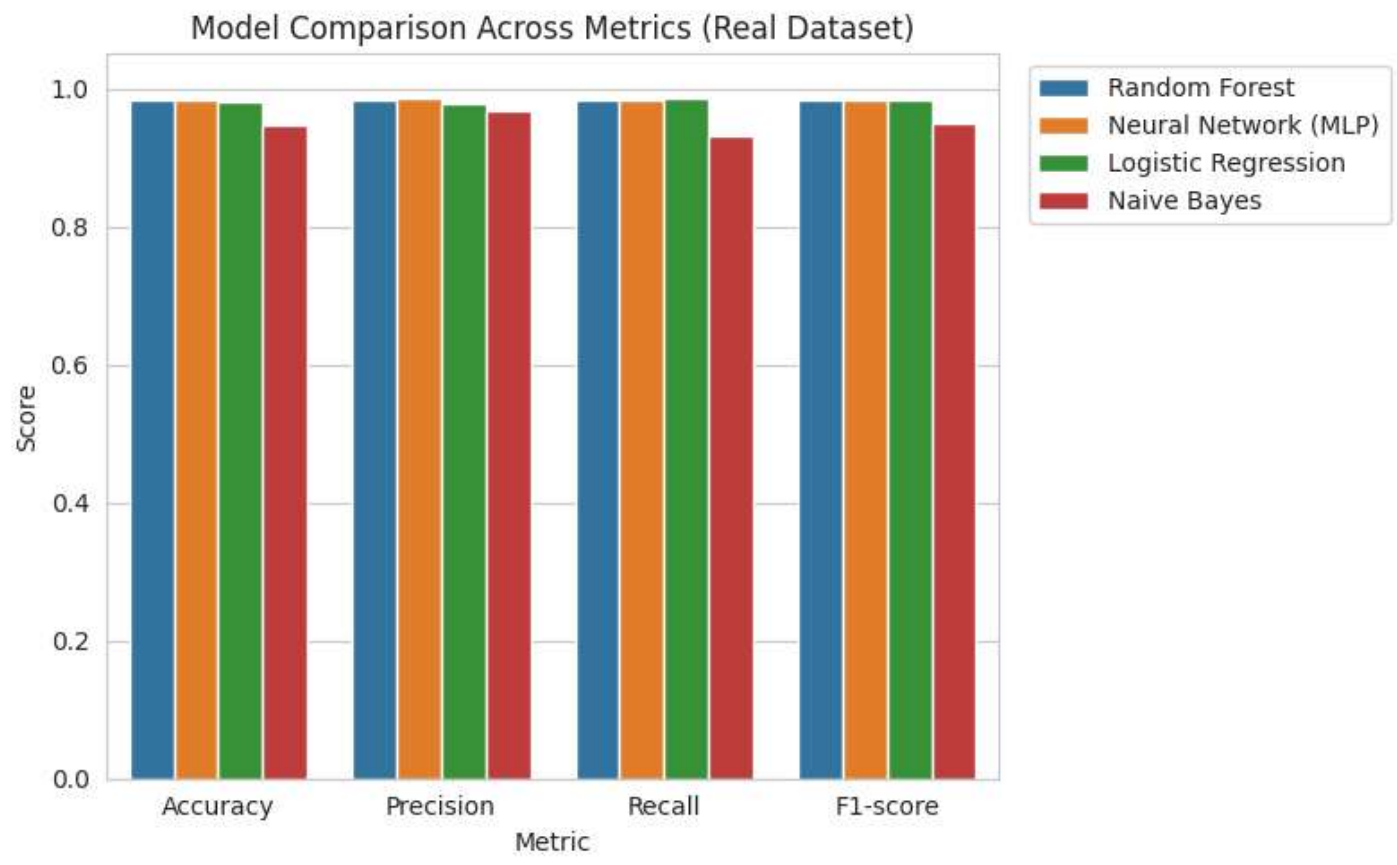
```
fig, axes = plt.subplots(2, 2, figsize=(11, 9))
axes = axes.flatten()

for ax, (name, preds) in zip(axes, predictions.items()):
    cm = confusion_matrix(y_test, preds)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax,
                xticklabels=["legitimate", "phishing"],
                yticklabels=["legitimate", "phishing"])
    ax.set_title(name)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

plt.tight_layout()
plt.savefig("confusion_matrices_real_data.png", dpi=150)
plt.show()
```




```
rf_model = trained_models["Random Forest"]
importances = rf_model.feature_importances_
top_n = 20
top_idx = np.argsort(importances)[-top_n:]

plt.figure(figsize=(8, 7))
plt.barh([feature_names[i] for i in top_idx], importances[top_idx], color="tomato")
plt.xlabel("Importance")
plt.title(f"Top {top_n} Most Important Features (Random Forest)")
plt.tight_layout()
plt.savefig("feature_importance_real_data.png", dpi=150)
plt.show()
```




```
plt.figure(figsize=(8, 5))
melted = results_df.melt(id_vars="Model", var_name="Metric", value_name="Score")
sns.barplot(data=melted, x="Metric", y="Score", hue="Model")
plt.ylim(0, 1.05)
plt.title("Model Comparison Across Metrics (Real Dataset)")
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig("model_comparison_real_data.png", dpi=150)
plt.show()
```




```
def predict_new_email(email_text, model=None, model_name=None):
    if model is None:
        model_name = model_name or best_model_name
        model = trained_models[model_name]

    cleaned = clean_text(email_text)
    meta = extract_metadata_features(email_text)

    text_vec = tfidf.transform([cleaned])
    meta_vec = csr_matrix(meta[meta_cols].values.astype(float).reshape(1, -1))
    combined = hstack([text_vec, meta_vec]).tocsr()

    pred = model.predict(combined)[0]
    proba_phishing = model.predict_proba(combined)[0][1] if hasattr(model, "predict_proba") else None
    label = "PHISHING" if pred == 1 else "LEGITIMATE"
```

```
        if proba_phishing is not None:
            confidence = proba_phishing if pred == 1 else (1 - proba_phishing)
            return f"{label} (confidence: {confidence:.2%})"
        return label

    print(predict_new_email(
        "urgent your account has been suspended click here http www secure-verify-login com "
        "to confirm your identity immediately or lose access"
    ))
```